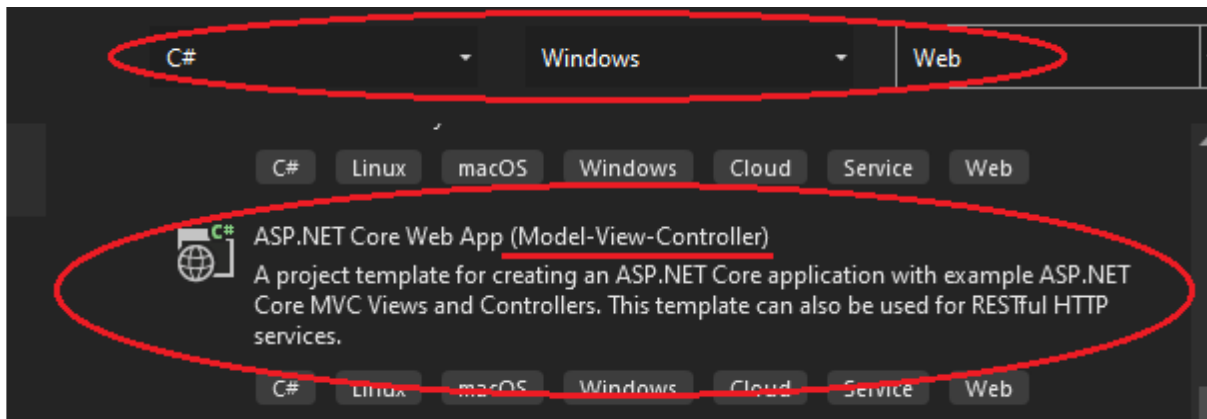


ASP.NET Core Crash Course - C# App in One Hour

VS2022 + .NET 5.0 (current)



Project name

JokesWebApp

Location

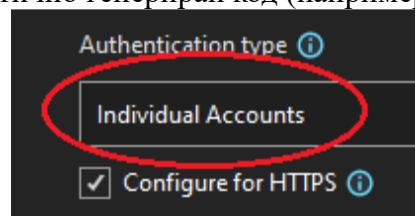
E:\VisualStudioCrashCourse

Solution name ⓘ

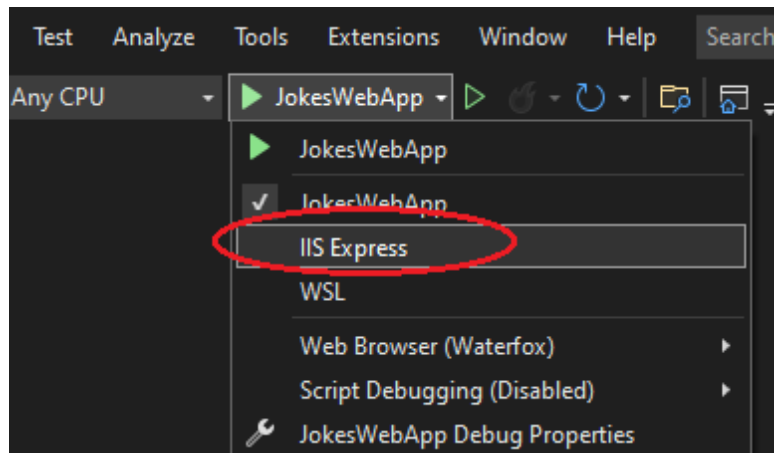
JokesWebApp

☒ Place solution and project in the same directory

За да има максимално автоматично генериран код (например папка Migrations в Data):

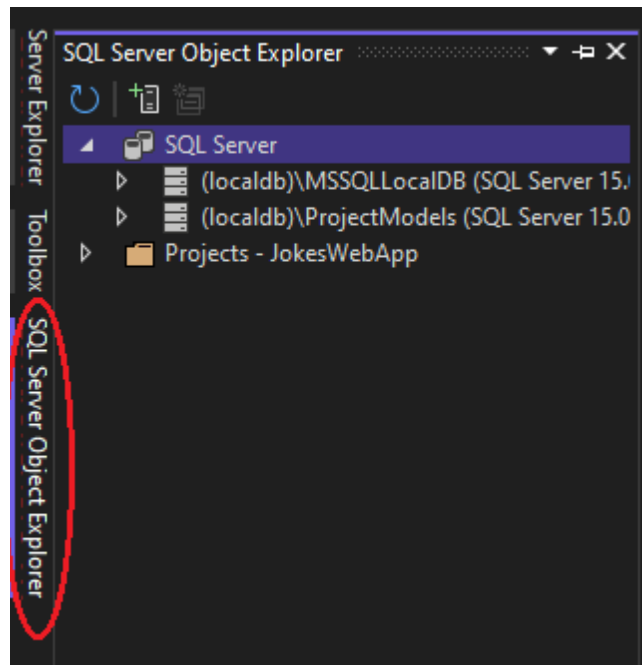


IIS Express = Information Internet Serrver (Microsoft Web Server)



<https://localhost:44388/>

.cshtml = c# html – комбинация от 2 езика, блендирани (смесени) в 1 страница.



Models Views and Controllers (MVC)

- Model = classes (objects)
- View = web page (Razor HTML)
- Controller = connects models, business logic and web pages.

Модел-Изглед-Контролер (Model-View-Controller или MVC) е архитектурен [шаблон за дизайн](#) в програмирането, основан на разделянето на бизнес логиката от графичния интерфейс и данните в дадено приложение.

Модел – ядрото на приложението, обикновено това са данните от реалния свят, които се моделират и над които се работи.

• **Изглед** (англ. View) – тази част от изходния код на приложението (html-сайт), отговорна за показването на данните от модела.

• **Контролер** – тази част от сорс кода (клас или библиотека), която взима данните от модела или извиква допълнителни методи върху модела, предварително обработва данните, и чак след това ги дава на изгледа.

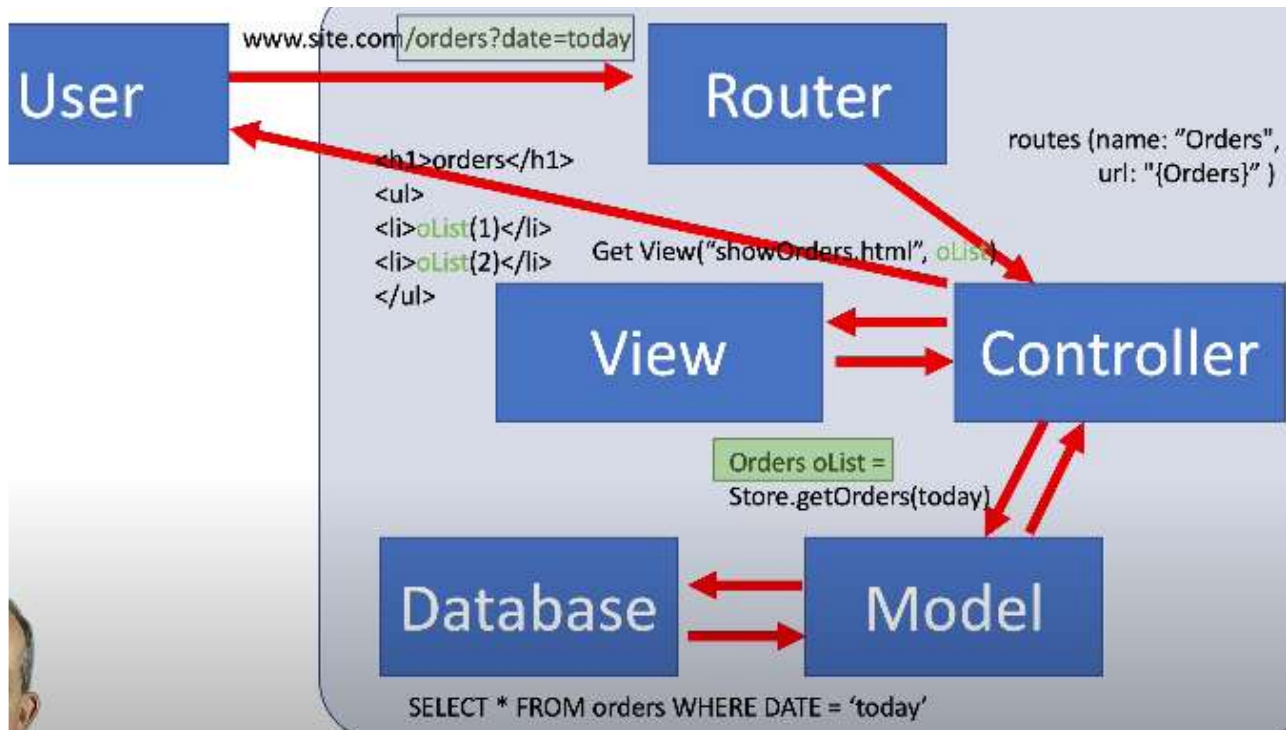
• Моделът е независим от контролера и изгледа.

• Моделът може да бъде планиран и осъществен независимо от другите части на системата.

• За един и същи модел могат да бъдат осъществени различни изгледи (интерфейси)

• Контролерът и изгледът могат да бъдат променени, без да се налага промяна в модела.

Модел = статична схема; контролера е динамичното действие;



```

public IActionResult Index()
{
    return View();
}

0 references
public IActionResult Privacy()
{
    return View();
}

[ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
0 references
public IActionResult Error()
{
    return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifie

```

Solution Explorer shows the project structure: JokesWebApp (1 of 1) with folders: Connected Services, Dependencies, Properties, wwwroot, Areas, Controllers, Data, Models, Views. The **HomeController.cs** file is selected in the Controllers folder.

Error – ако кликнеш на линк, който не съществува.



View

- HTML CSS code (or similar)
- Usually gets a list of data from the controller.
- Dynamically combines data with HTML in a template

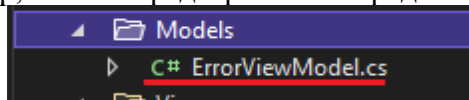
Razor (бръснач) – езикът, работната среда, технологията.



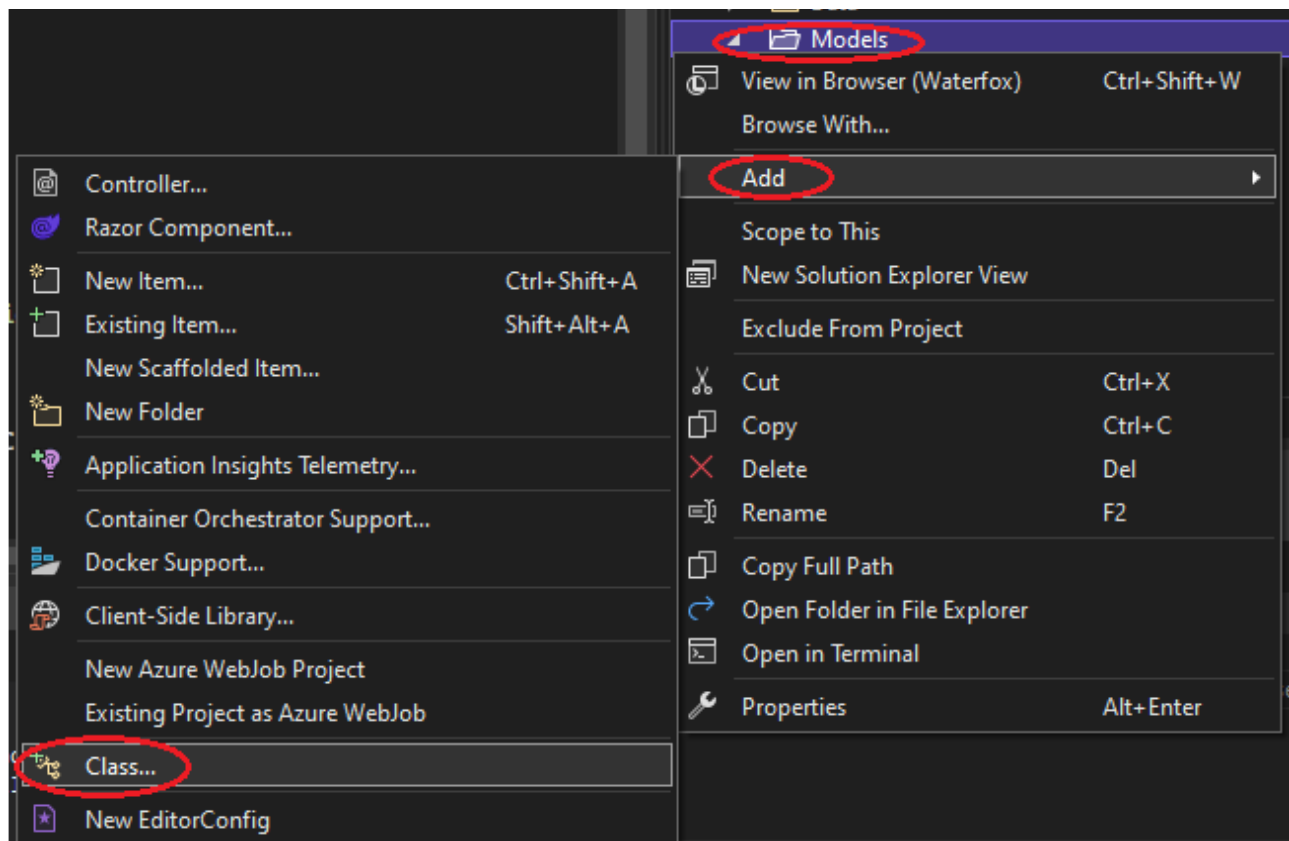
Model

- Data Related
- Consists of classes / objects with properties.
- Uses SQL statements.
- Supplies the controller with lists of objects.

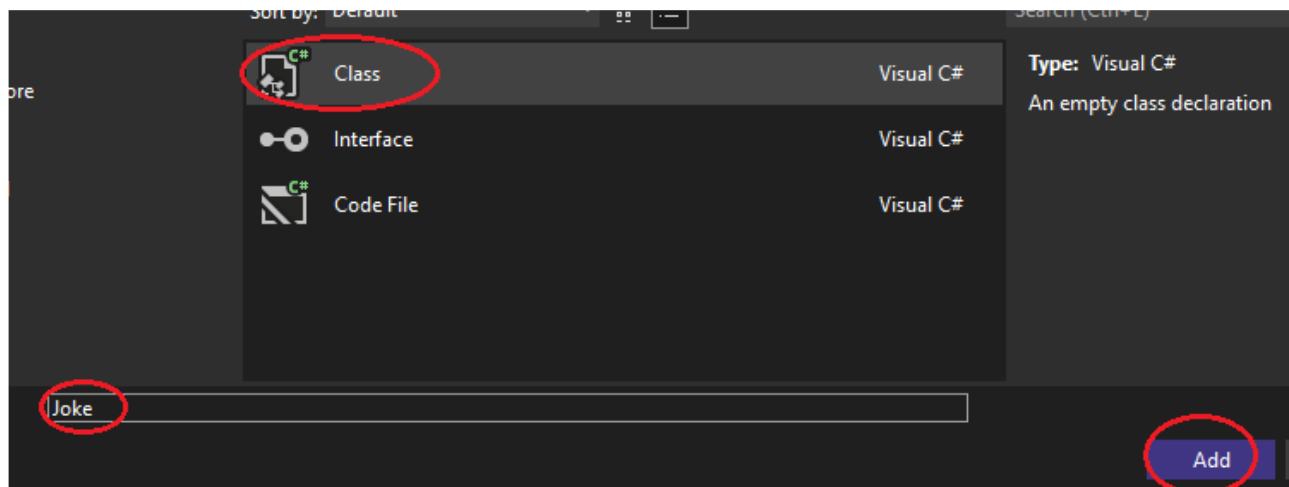
За по-лесен нагледен пример, вместо предварително вградения `ErrorViewModel.cs`



ще добавим наш модел.

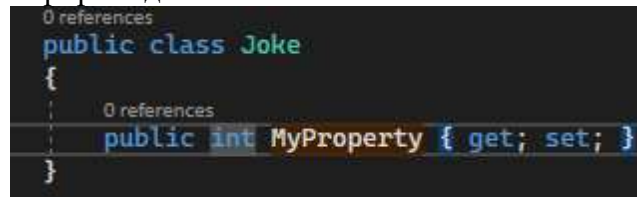


Вместо Class.cs пишем Joke (без расширения).

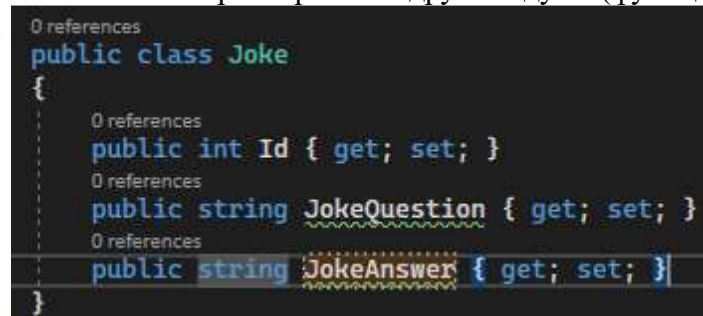




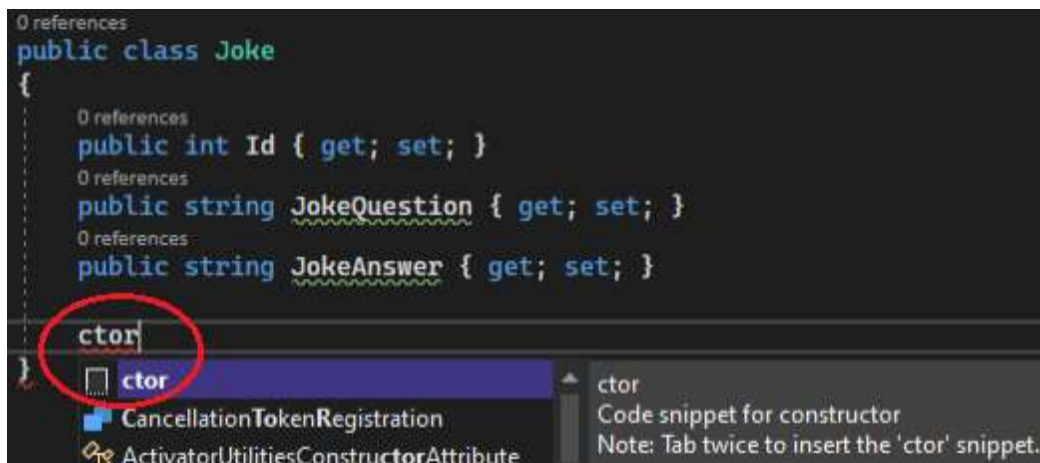
Tab 2x – автоматично генерира код.



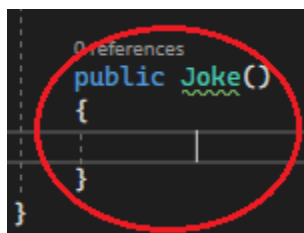
Със следващите единични Tab се прехвърля към другите думи (функции)



Следва въвеждане на конструктор ctor, с който да могат да работят функциите:

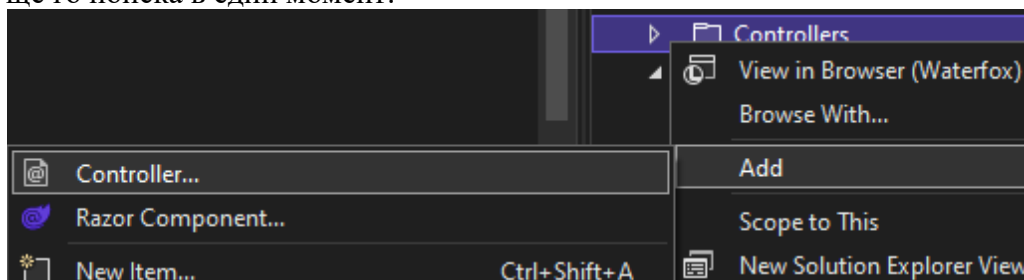


конструктор Joke – автоматично генериран



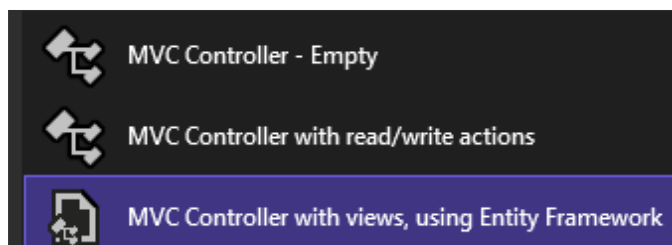
Засега конструкторът се оставя празен, защото ще се използва от други парчета код и е необходимо да остане празен.

Автоматично генериран файл ApplicationDbContext.cs. Трябва да се уверим, че е там, защото ще го поиска в един момент:

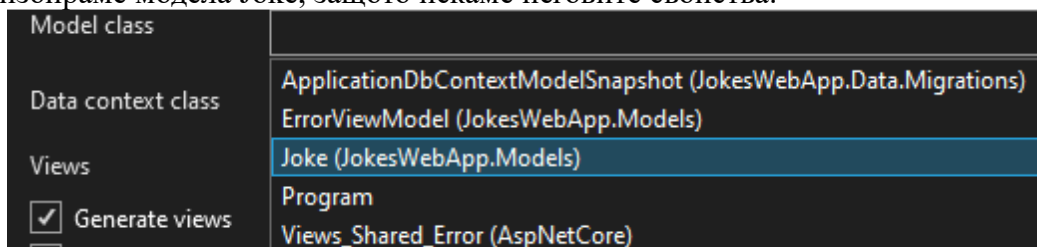


Автоматично ще генерираме много код:

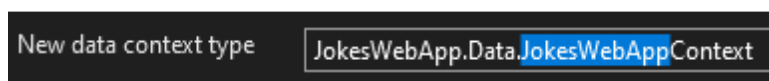
Отново изборът с най-много авто-код за този интензивен туториъл (чете, записва, трие, редактира):



После избираме модела Joke, защото искаме неговите свойства:



Променяме името, така че да уеднаквим с името на файла в Migrations папката, което видяхме преди малко:



New data context type

Add

Става

Уверяваме се, че всички отметки са маркирани, съответно ще има много автоматично генериран код и по-голямо забавяне.

Add MVC Controller with views, using Entity Framework

Model class

Data context class

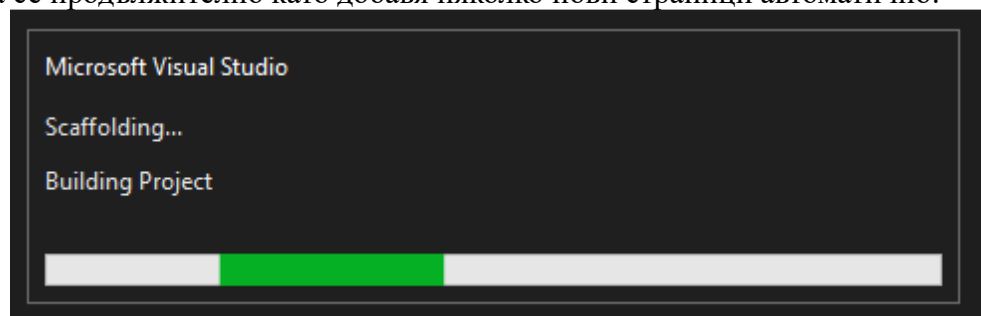
Views

- ☒ Generate views
- ☒ Reference script libraries
- ☒ Use a layout page

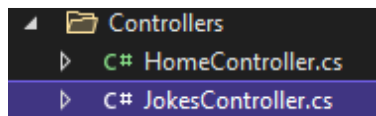
Не успях да направя навреме скрийншот, но автоматично ни дописва и името на контролера QuotesController -> OK

Controller name

Генерира се продължително като добавя няколко нови страници автоматично:



Една от новостите е JokesController.cs



И в папка Views има нова папка Jokes, съответно с 5 нови страници:

единственият обект.

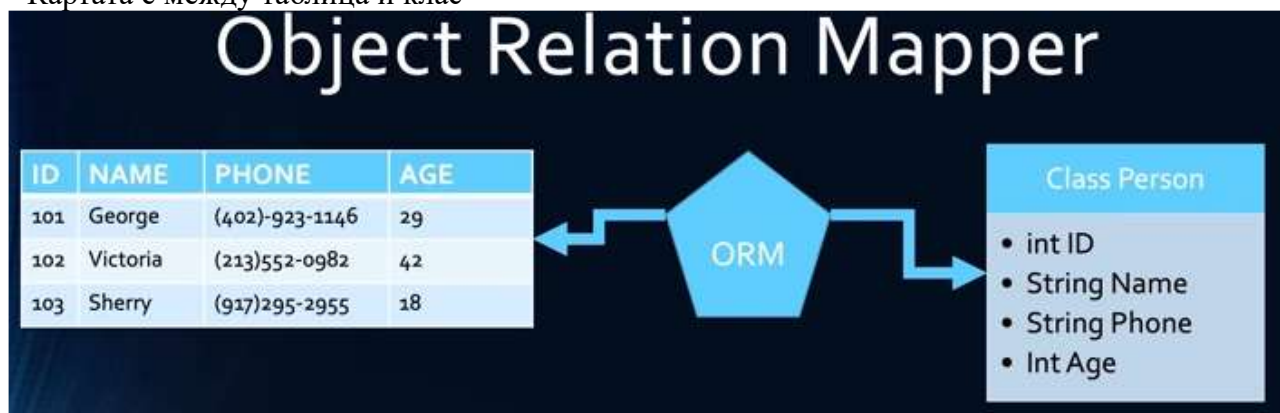
```
migrationBuilder.CreateTable(  
    name: "Joke",
```

С колони JokeQuestion, JokeAnswer

```
.Annotation("SqlServer:Identity", 1, 1),  
    JokeQuestion = table.Column<string>(type: "nvarchar(max)", nullable: false),  
    JokeAnswer = table.Column<string>(type: "nvarchar(max)", nullable: false)
```

Така че, папка Migrations е езикът на създаване на таблици.

Картата е между таблица и клас



Има два начина за създаване на БД за система като тази DAO и ORM:

DAO (data access object):

*традиционен, старомоден с ръчно въвеждане и все още предпочитан от компаниите и програмистите, заради по-голямата видимост какво се случва в БД и съответно по-големия контрол;

ORM (object relation mapper):

* предпочитан от програмистите, когато искат да спестят време; автоматично генерира таблици без да се налага да пишат никакви SQL-твърдения/отчети; автоматично генерира таблици въз основа на дефинирани класове в приложенията; БД се обновява чрез миграциите; подходящи за **прости** приложения.

Entity – името на ORM на Майкрософт, който използваме в момента;

Ще направим 2 таблици: една за потребителските акаунти, другата за шегите.

Database Setup

DATA ACCESS OBJECT (DAO)

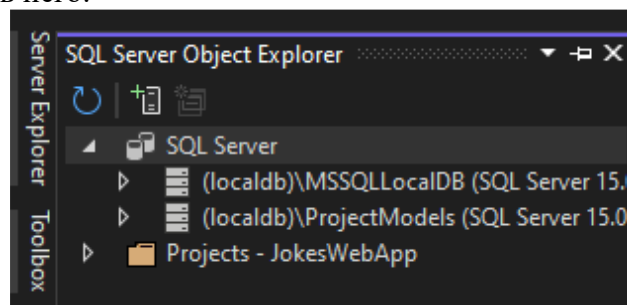
- Manually create tables.
- Traditional method of database access.
- Write your own SQL statements.
- Database managers (DBA's) usually prefer DAOs.

Provides more visibility on finding problems.

OBJECT RELATIONAL MAPPER (ORM).

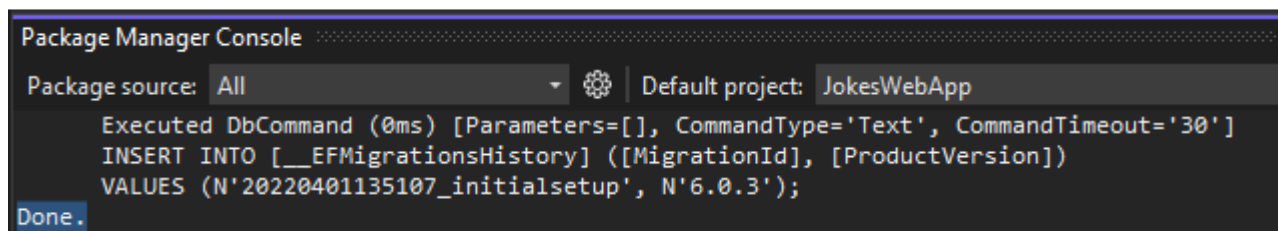
- Allow the computer to generate database tables based on classes defined in the application.
- Database is updated using migrations.
- **Entity Framework** is Microsoft's ORM.
- Simple for basic applications.

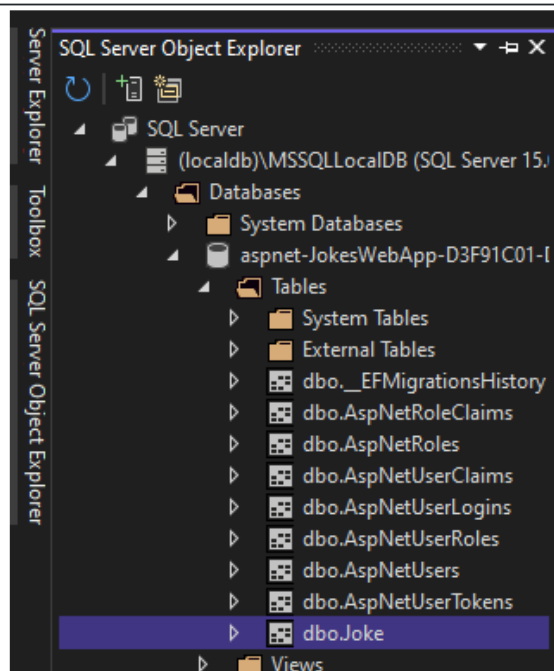
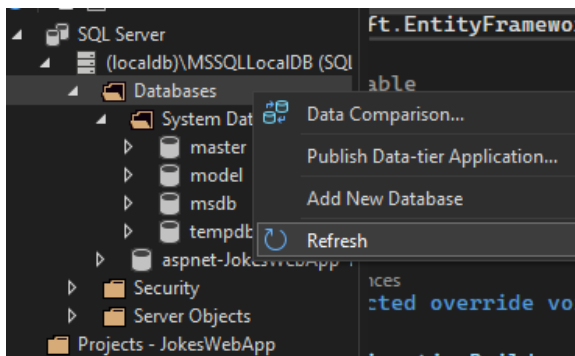
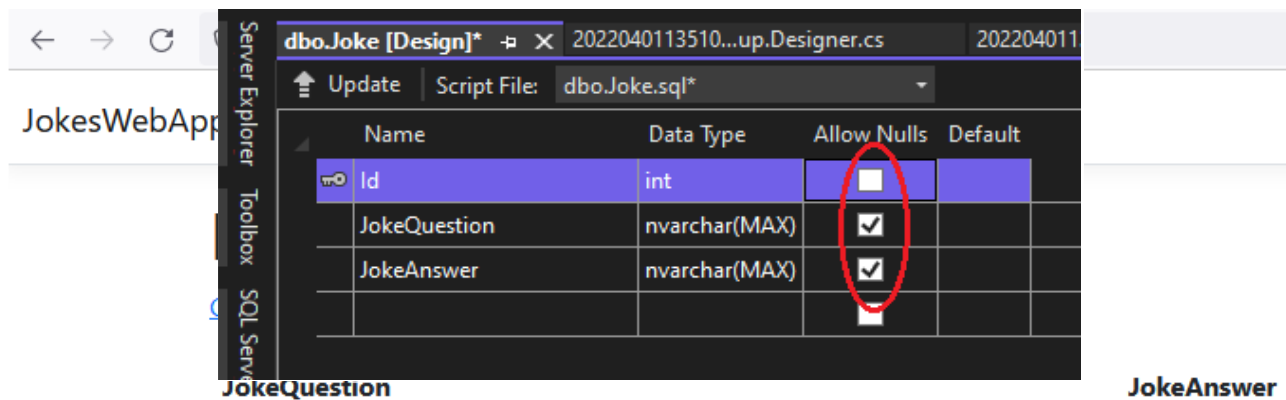
Превключваме в SQL Object Explorer (ако го няма, изкарваме от View) и виждаме 2 различни проекта БД в него:



Връщаме се в Package Manager:

PM> update-database





Най-важното до тук е крeйтoра в мoдeлa: Models->Add->Class “Joke”
[namespace JokesWebApp_new.Models](#)

```

{
    public class Joke
    {
        public int Id { get; set; } // prop
        public string JokeQuestion { get; set; }
        public string JokeAnswer { get; set; }

        public Joke() // ctor
        {
        }
    }
}

```

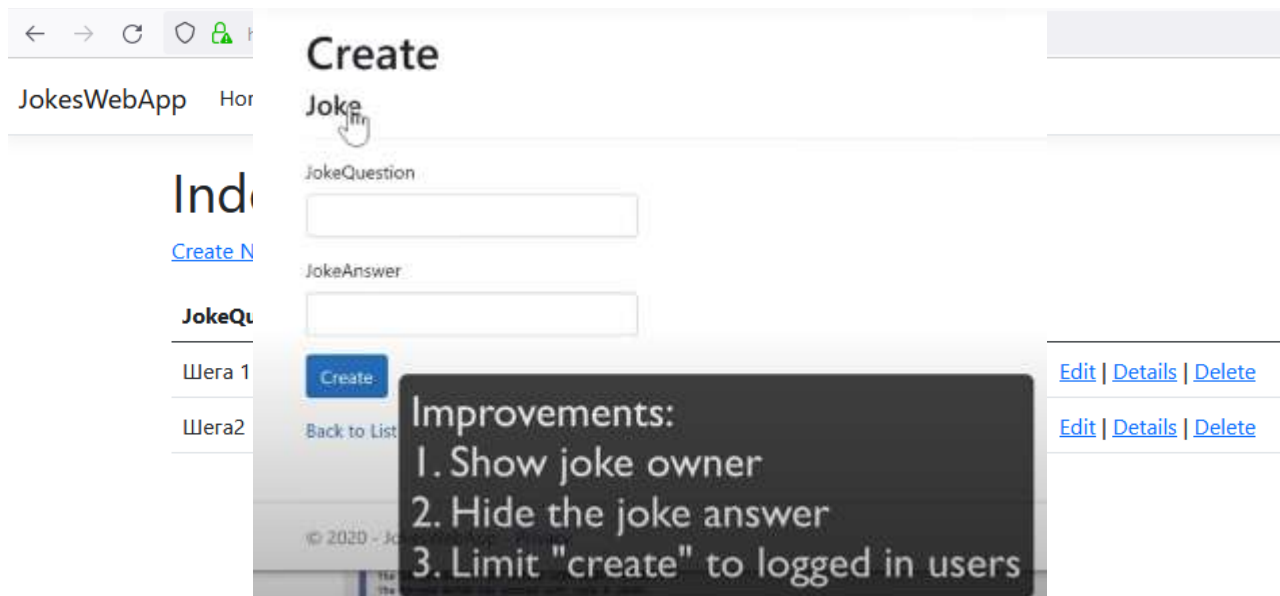
*
 Автоматично генериран файл Data->Migrations->ApplicationDbContext.cs. Трябва да се уверим, че е там, защото ще го поиска в един момент:
 Controllers->Add->Controller->MVC Controller with Views.... → ApplicationDbContext
 *

```

PM> add-migration "initsetup"
PM> update-database

```

*
 SQL server object explorer->



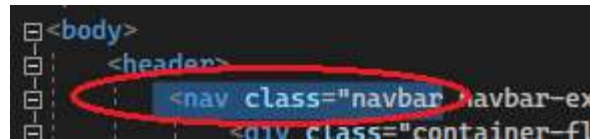
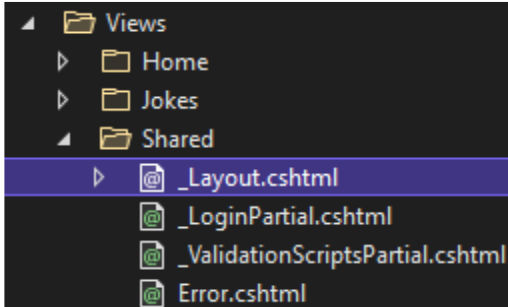
Дотук: в модела добавяме клас;

- 1) в миграциите добавяме контролер;
- 2) в РМ за класа създаваме миграция към база данни.
- 3) В layout – navigationbar с препратка към метод ShowSearchForm
- 4) В контролера JokesController.cs добавяме ShowsearchForm и автоматично View с д.б.
- 5) в генерирания ShowSearchForm.cshtml оформяме търсачката без валидации и модели, с търсене по SearchPhrase и действие action ShowSearchResults
- 6) вмъкваме филтър Where(j => j.JokeQuestion.Contains(SearchPhrase))
- 7) [Authorize] за редакции в контролера

- 8) wwwroot → site.css / ако не видим промените в брауъра Shift+Refresh и така обновяваме кеша на стила в брауъра.

Add item to Nav bar:

- Search



```
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
  <ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="" asp-controller="Jokes" asp-action="Index">Jokes</a>
    </li>
  </ul>
  <partial name="_LoginPartial" />
</div>
```

Прилича на html-css код.

What is Bootstrap class in C#?

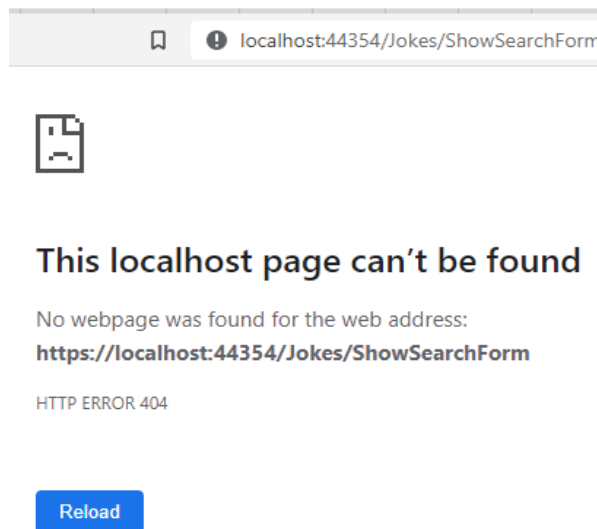
Bootstrap framework is **used to develop front facing web applications and sites**. ... The Bootstrap framework provides you all those components. The entire framework is module based, you can customize it with your own bit of CSS. It also provides JavaScript plugins for things like tooltips, popovers, modals and more.

За да се получи:

```
<ul class="navbar-nav flex-grow-1">
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
  </li>
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
  </li>
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Jokes" asp-action="Index">Jokes</a>
  </li>
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Jokes" asp-action="ShowSearchForm">Search</a>
  </li>
</ul>
```

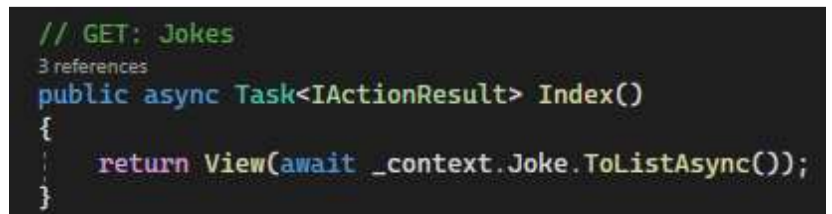
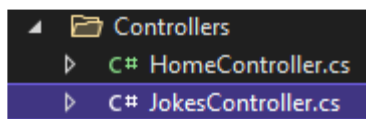

Където имаме 2 папки на сайта asp-controller съответно Home и Jokes; съответно във всяка от тях има по един Index файл (метод), а ние добавихме файл за търсачка.

Copy->Paste и сменяме вместо Index -> ShowSearchForm, която още не съществува, В URL адреса Jokes контролера (папката) съществува, но методът (файлът) ShowSearchForm не.

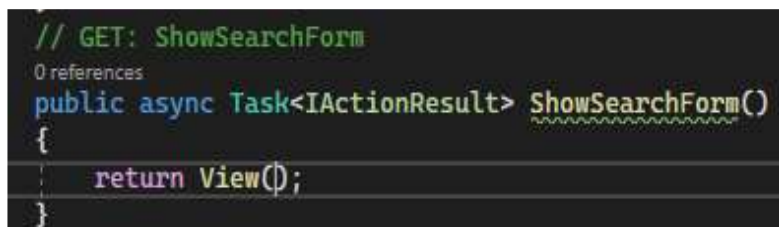


Папка Jokes съществува, но не и метод ShowSearchForm, съответно трябва да го създадем.

Единият начин е в папките да се въвеждат нови файлове с код, другият е автоматично с д.б. да се вмъкват шаблони с код.



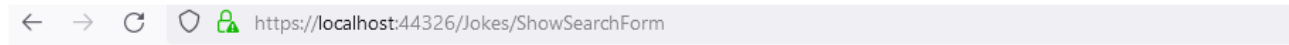
Copy->Paste и сменяме вместо Index → ShowSearchForm (когато някой напише този URL, това е, което ще се случи).



- Някакъв обикновен текст дава грешки.
`return View("Searching");`
- При пълно наименование на метода или нищо, не дава грешки.

```
return View("ShowSearchForm");
```

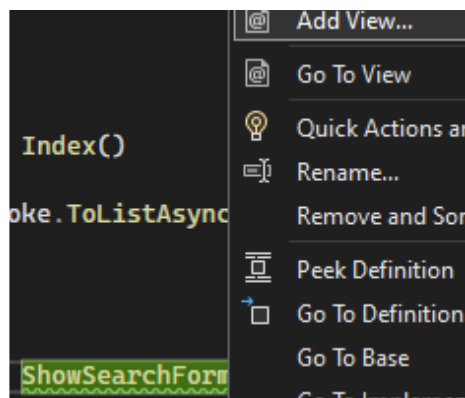
Следва чест отговор на грешка, когато правим приложение, но забравим да вмъкнем код, който извикваме. Показва, че такава страница не се открива, но има връзка с БД към нея, а само името ѝ и съответно се изброяват папките и файловете, в които е търсено автоматично.



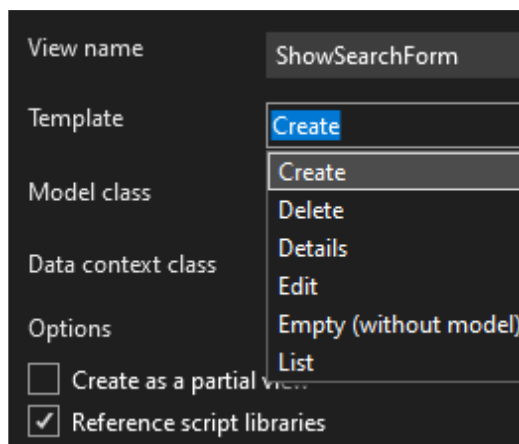
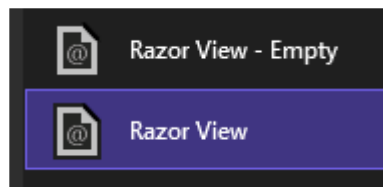
An unhandled exception occurred while processing the request.

InvalidOperationException: The view 'ShowSearchForm' was not found. The following locations were searched:
/Views/Jokes/ShowSearchForm.cshtml
/Views/Shared/ShowSearchForm.cshtml
/Pages/Shared/ShowSearchForm.cshtml

Microsoft.AspNetCore.Mvc.ViewEngines.ViewEngineResult.EnsureSuccessful(IEnumerable<string> originalLocations)



Затова с д.б. я създаваме



за максимален код, като най-близко до желанието ни е create, а не празна форма. WebAppJokes.Models е основният клас, с който работим

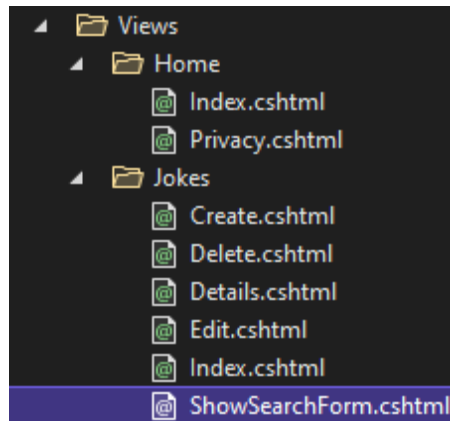
Model class	
Data context class	ApplicationDbContextModelSnapshot (WebAppJokes.Data)
Options	Joke (WebAppJokes.Models)
☐ Create as a partial view	Program
☐ Reference script libraries	Views_Jokes_Create (AspNetCore)

Options
☒ Create as a partial view
☒ Reference script libraries

Add Razor View

View name	ShowSearchForm
Template	Create
Model class	Joke (WebAppJokes.Models)
Data context class	ApplicationDbContext (WebAppJokes.Data)
Options	
☒ Create as a partial view	
☒ Reference script libraries	

partial view – част от друг код. Създава се нов файл – търсачка базирана на JokeQuestion и JokeAnswer.



```
<form asp-action="ShowSearchForm">
  <div asp-validation-summary="ModelOnly" class="text-danger">
  </div>
  <div class="form-group">
    <label asp-for="JokeQuestion" class="control-label">Question</label>
    <input asp-for="JokeQuestion" class="form-control" type="text" />
    <span asp-validation-for="JokeQuestion" class="text-danger"></span>
  </div>
  <div class="form-group">
    <label asp-for="JokeAnswer" class="control-label">Answer</label>
    <input asp-for="JokeAnswer" class="form-control" type="text" />
    <span asp-validation-for="JokeAnswer" class="text-danger"></span>
  </div>
</form>
```

Joke

JokeQuestion

JokeAnswer

Create

[Back to List](#)

Изтриваме ID и JokeQuestion, а на мястото на Create → Search (видим текст на бутона)

```
<form asp-action="ShowSearchForm">
  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
  <div class="form-group">
    <label asp-for="JokeAnswer" class="control-label"></label>
    <input asp-for="JokeAnswer" class="form-control" />
    <span asp-validation-for="JokeAnswer" class="text-danger"></span>
  </div>
  <div class="form-group">
    <input type="submit" value="Search" class="btn btn-primary" />
  </div>
</form>
```

Joke

JokeAnswer

Search

[Back to List](#)

Показва се, но не се обработва резултат. Моделът на ред 1 се отстранява, защото не искаме

да създаваме нови шапки, а търсачка.

```
25 @section Scripts {
26     @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
27 }
```

Изтриваме излишното на последните редове:

Получаваме 3 грешки в центъра на формата:

```
<form asp-action="ShowSearchForm">
  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
  <div class="form-group">
    <label asp-for="JokeAnswer" class="control-label"></label>
    <input asp-for="JokeAnswer" class="form-control" />
    <span asp-validation-for="JokeAnswer" class="text-danger"></span>
  </div>
```

Те са, защото изтрихме модела в началото на кода, който казва, че страницата е Joke, а вече няма никакви Jokes. Сменяме `<input asp-for="JokeAnswer" class="form-control" />` с `<input name="SearchPhrase" class="form-control" />` и над него `<label for="SearchPhrase" class="control-label"></label>`

, което е готова форма като стандарт, стига имената на извиканата SearchPhrase да са еднакви. Премахваме и 2 реда с валидация под и над SearchPhrase

`<span asp-validation-for="JokeAnswer" class="text-danger"></span>`

Не искаме в началото формата за търсене да се показва сама, затова създаваме ново действие `ShowSearchResults`

```
<form asp-action="ShowSearchResults">
  <div class="form-group">
    <label for="SearchPhrase" class="control-label"></label>
    <input name="SearchPhrase" class="form-control" />
  </div>
  <div class="form-group">
    <input type="submit" value="Search" class="btn btn-primary" />
  </div>
```

`<h4>Search for a joke</h4>`

`<hr />`

`<div class="row">`

`<div class="col-md-4">`

`<form asp-action="ShowSearchResults">`

`<div class="form-group">`

`<label for="SearchPhrase" class="control-label"></label>`

`<input name="SearchPhrase" class="form-control" />`

`</div>`

`<div class="form-group">`

`<input type="submit" value="Search" class="btn btn-primary" />`

`</div>`

`</form>`

`</div>`

`</div>`

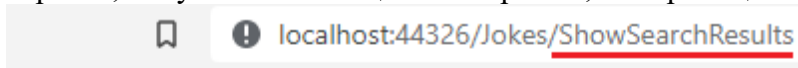
```
<div>
    <a asp-action="Index">Back to List</a>
</div>
```

Search for a joke

Search

[Back to List](#)

Когато опитаме търсене, получаваме съобщение за грешка, че страницата не е намерена.



This localhost page can't be found

No webpage was found for the web address:

https://localhost:44326/Jokes/ShowSearchResults

HTTP ERROR 404

За да я създадем, отиваме отново в JokesController.cs

Последните промени са върху ShowSearchForm, съответно Copy->Paste

```
// GET: ShowSearchForm
public async Task<IActionResult> ShowSearchForm()
{
    return View();
}
```

```
// GET: Jokes
3 references
public async Task<IActionResult> Index()
{
    return View(await _context.Joke.ToListAsync());
}
```



```
// GET: ShowSearchForm
0 references
public async Task<IActionResult> ShowSearchForm()
{
    return View();
}

// GET: ShowSearchForm
0 references
public async Task<IActionResult> ShowSearchForm()
{
    return View();
}
```

Като сменяме GET → PoST и метод ShowSearchResults(), защото не искаме ShowSearchForm да се показва сама, а искаме да отидем към друг обект ShowSearchResults.

С тестова цел:

```
// PoST: Jokes/ShowSearchResults
0 references
public async Task<IActionResult> ShowSearchResults(string SearchPhrase)
{
    return "You searched: " + SearchPhrase;
}

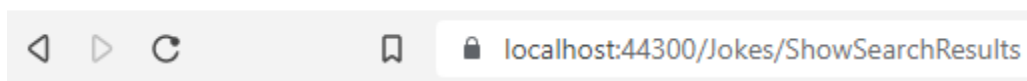
// View("Index", await ShowSearchResults(SearchPhrase))

string string.operator +(string left, string right)
CS0029: Cannot implicitly convert type 'string' to 'Microsoft.AspNetCore.Mvc.IActionResult'
```

```
// PoST: ShowSearchResults
public string ShowSearchResults(String SearchPhrase)
{
    return "You searched: " + SearchPhrase;
}
```

Като с червена линия е подчертана грешката, която при посочване се изписва като IActionResult, защото не може един тип данни да се конвертира в друг (например int → string) съответно се връщаме в по-горния ред и правим корекции:

```
// PoST: Jokes/ShowSearchResults
0 references
public string ShowSearchResults(string SearchPhrase)
{
    return "You searched: " + SearchPhrase;
}
```



You searched: cow

С демонстративна цел разбрахме, че данните, които са входирани и получени от контролера, след което изпратени към следващия обект на страницата, т.е. Всичко е коректно записано във формата.


```
// PoST: Jokes/ShowSearchResults
0 references
public async Task<IActionResult> ShowSearchResults(string SearchPhrase)
{
    return View(await _context.Joke.ToListAsync());
}
```

Това ще изкара всички резултати в базата данни.

```
// PoST: ShowSearchResults
0 references
public async Task<IActionResult> ShowSearchResults(String SearchPhrase)
{
    return View("Index", await _context.Joke.ToListAsync());
}
```

Искаме филтрирани резултати. За целта имаме готов модел:

```
return View(await _context.Joke.ToListAsync());
```

```
// PoST: Jokes/ShowSearchResults
0 references
public async Task<IActionResult> ShowSearchResults(string SearchPhrase)
{
    return View("Index", await _context.Joke.Where( j => j.JokeQuestion.Contains(SearchPhrase)).ToListAsync());
}
```

В този вид продължава да изкарва всички резултати в БД, дори когато вкарваме дума, която не се намира в нея.

Реално, филтърът на търсачката е този вмъкнат
Where(j => j.JokeQuestion.Contains(SearchPhrase))

От Jokes/Index.cshtml искаме да премахнем JokeAnswer, за да се вижда само при Details

```
@model IEnumerable<WebApplication2.Models.Joke>
```

IEnumerable – лист

Искаме само логнат потребител да може да добавя шеги, така че преди Create в JokeController.cs трябва да добавим декоратор [Authorize], който се подчертава като грешка с червена вълнообразна линия, а при клик на балончето се предлага поправка на грешката с добавяне на библиотека за нея в началото на файла, с което действието приключва.

```

56
57 [Authorize]
58 // GET: Jokes/Create
0 references
59 public IActionResult Create()
60 {
61     return View();
62 }

```

```

[Authorize]
// GET: Jokes/Create
0 references
public IActionResult Create()
{
    using Microsoft.AspNetCore.Authorization;
    Microsoft.AspNetCore.Authorization.Authorize
    Generate type 'Authorize'
    Use expression body for methods
    // POST: Jokes/Create
    // To protect from overposting attacks,
    // For more details, see http://go.micro

```

CS0246 The type or namespace name 'AuthorizeAttribute' could not be found (are you missing a using directive or an assembly reference?)

Lines 5 to 6

```

using System.Threading.Tasks;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

```

```

57
58 [Authorize]
59 // GET: Jokes/Create

```

```

4 using System.Linq;
5 using System.Threading.Tasks;
6 using Microsoft.AspNetCore.Authorization;
7 using Microsoft.AspNetCore.Mvc;

```

С което вече за писане на нова шега се изисква log in.

За авторизация дори е по-важно поставянето/влизането в БД

```

65 // POST: Jokes/Create
66 // To protect from overposting attacks, enable the specific properties you want to bind to
67 // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
68 [Authorize]
69 [HttpPost]
70 [ValidateAntiForgeryToken]
0 references
71 public async Task<IActionResult> Create([Bind("Id,JokeQuestion,JokeAnswer")] Joke joke)

```

По същият начин авторизираме всичките //Get и //PoST на Create, Edit, Delete.